

LOW-LEVEL DESIGN • BEGINNER-FRIENDLY, NOTHING SKIPPED

The LLD/OOD Interview Handbook

Starts from "what is a class?" and builds all the way up to five complete, worked case studies with runnable C++ code — nothing assumed, nothing skipped. Every definition is explained in plain language with a small example before any interview question uses it. Categories, question selection, and case studies were dictated by current interview-report and practitioner sources, not a fixed syllabus. A companion single-file **LaTeX edition** of this same guide is provided separately so you can compile your own PDF with no editing required.

Zero assumed knowledge → 5 full case studies

Code skeletons in C++

Mermaid class & sequence diagrams

Sourced from Hello Interview, GfG, Glassdoor, Blind, ashishps1/awesome-low-level-design

Updated Sep 2026

WHO THIS IS FOR & TIME MANAGEMENT

Intern

Junior

Mid

Senior

- **Intern/Junior (0–2 YOY)** — LLD/OOD is your primary design round; expect Parking Lot, Tic-Tac-Toe-style problems and direct SOLID/pattern questions.
- **Mid (2–5 YOY)** — expect Elevator, LRU Cache, Rate Limiter with concurrency follow-ups layered on.
- **Senior+** — LLD is usually replaced by system design, but still appears as a warm-up or for infra-adjacent roles (concurrency-heavy problems).

A 45–60 min interview, budgeted: 5–8 min requirements clarification → 5 min core entities/responsibilities → 15–20 min class diagram + key methods → 10–15 min code skeleton for the trickiest part → 5–10 min follow-ups/tradeoffs. Ask upfront how much actual code vs. talk-through the interviewer wants — sources are explicit this varies by company and changes your pacing.

WHAT INTERVIEWERS EVALUATE

- **Requirements-to-classes translation** — turning a fuzzy prompt into clean entities without over- or under-modeling.
- **SOLID & pattern judgment** — applied to a specific class with a named violation scenario, not recited abstractly.
- **Communication** — narrating trade-offs out loud; interviewers cannot read your mind.
- **Extensibility under a changing prompt** — the interviewer will move your goalposts mid-interview; how you adapt the design is the actual signal.

00 How LLD Interviews Work

Before any theory: what actually happens in the room (or on the call), what the interviewer is quietly scoring, and how to spend your limited minutes on purpose instead of by accident.

What is an LLD interview, in plain terms?

An interviewer describes a real-world system in one or two sentences — "design a parking lot," "design a vending machine" — and asks you to turn that into actual classes: what objects exist, what each one is responsible for, how they talk to each other, and (often) working code for the trickiest part. There is deliberately no single "correct" diagram. The interviewer already knows several reasonable designs; what they're actually watching is *how you get there* — the questions you ask, the trade-offs you notice, and whether your design holds together when they change the rules on you halfway through.

What interviewers are quietly scoring

THEY'RE WATCHING FOR...	IN PLAIN TERMS, THIS MEANS
Requirements-to-classes translation	Can you turn a vague sentence into a short, sensible list of classes — not too few (one giant class doing everything) and not too many (a class for every noun you can think of)?
Naming and structure	Do your class names, method names, and relationships read clearly to <i>someone else</i> , or only make sense in your own head?
Justifying choices out loud	Do you say <i>why</i> you picked a hash map here, an interface there — or do you just draw boxes silently?
Handling a changed requirement	When the interviewer adds "now also support X," do you extend your design, or does everything fall apart and need a redo?

Time-boxing a 45-60 minute interview

```
0:00 - 0:05  Clarify requirements (Section "the framework", Step 1) – ask questions, don't assume
0:05 - 0:10  List the core classes and what each one is responsible for
0:10 - 0:15  Draw the class diagram (boxes + arrows), narrating as you go
0:15 - 0:30  Write the key method signatures, then code the trickiest one fully
0:30 - 0:40  Interviewer asks a follow-up ("now add feature X") – extend, don't restart
0:40 - 0:45  Mention tests, edge cases, and trade-offs you didn't have time to fully explore
```

If you only have **30 minutes**, compress the clarification and diagram steps and spend the majority of remaining time on one well-implemented class rather than a shallow pass over everything. If you have **90 minutes**, expect two full case-study-style rounds, or one case study pushed much further (concurrency, a second major feature, a full test suite) — the extra time is testing depth, not breadth.

How to ask good clarifying questions (a beginner's script)

If you've never done this before, it can feel awkward to "ask questions instead of just starting." Here is a simple, reusable script — you can say almost exactly this in any LLD interview:

- **"What are the core actions the system needs to support?"** — pins down the functional requirements before you design anything.
- **"Roughly what scale are we talking about — one of these, or millions?"** — a single parking garage and a citywide network of them are different designs entirely.
- **"Is there anything you'd like me to explicitly leave out of scope for now?"** — this single question prevents you from wasting time gold-plating a part nobody asked for.

- **"Would you like me to write real, compilable code, or is a class diagram plus pseudocode enough?"** — different interviewers and companies expect very different levels of code completeness; asking this upfront directly controls your own pacing.

BEGINNER REASSURANCE

Asking these questions is not a sign of weakness — every source used for this guide agrees it's exactly what separates a stronger answer from a weaker one. A candidate who starts drawing boxes immediately, with no questions asked, is a documented weak signal, not a confident one.

WHAT THE WEB SOURCES EMPHASIZE

Current sources describe LLD prompts as "intentionally open-ended and unstructured, mirroring real-life conditions" specifically so that the clarifying-question step itself is part of what's graded — this is why Section 0 exists as its own section here, ahead of any theory.

01 OOP Fundamentals, SOLID & Core Design Principles

Every source agrees this is the entry gate: encapsulation, inheritance, polymorphism, abstraction, and the five SOLID principles. This section starts from zero — if you already know what a class is, skip to 1.1. What separates a strong answer at every level, per multiple sources independently, is mapping each idea to a specific class with a named example or violation — not reciting a definition.

CORE DEFINITIONS

Object-Oriented Design (OOD)

The activity of deciding what classes/objects a system needs, what each is responsible for, and how they collaborate — the output is a class diagram and key method signatures, not running infrastructure.

Low-Level Design (LLD)

OOD applied concretely enough to produce compilable code — interviewers use the terms near-interchangeably, though LLD leans slightly more toward "can you actually implement this," per Naukri/Code360's framing.

1.0 BASIC Absolute basics: what is a class, and what is an object?

Think of a **class** as a blueprint, and an **object** as one real thing built from that blueprint. A blueprint for a house isn't a house you can live in — it's a plan describing what every house built from it will have (a number of rooms, a door, windows). A class works the same way: it describes what data (called **attributes** or **fields**) and what actions (called **methods**) every object made from it will have.

```
class Dog {
public:
    Dog(std::string name) : name_(name) {}          // builds one Dog object
    void bark() { std::cout << name_ << " says woof!"; }
private:
    std::string name_;                             // an attribute – data the object holds
};

Dog myDog("Rex");      // myDog is an OBJECT – one specific, real Dog built from the Dog class
Dog yourDog("Bella");  // a second, completely separate object from the same blueprint
myDog.bark();          // "Rex says woof!" – calling a METHOD, an action the object can do
```

Dog is the class — it exists once, in your source code, as a description. myDog and yourDog are two separate objects — each has its own copy of the name_ attribute (one holds "Rex," the other "Bella"), even though both were built from the exact same class. This is the single most foundational idea in all of object-oriented design: **a class describes; an object exists**. Every other term in this section — encapsulation, inheritance, polymorphism — is really just a rule about how classes and objects are allowed to relate to each other.

class object attribute/field method

1.1 BASIC What are the four pillars of OOP, with a concrete example of each?

- **Encapsulation** — bundling data with the methods that operate on it, and hiding internal state behind a controlled interface. A ParkingSpot class exposes occupy()/vacate() rather than letting external code flip a public bool isOccupied directly — the class alone enforces its own invariants.
- **Abstraction** — exposing only what a caller needs, hiding how it's done. A PaymentProcessor interface exposes charge(amount); callers never see whether it's calling Stripe or a mock.
- **Inheritance** — a class acquiring the interface/behavior of a more general class. A Car and a Truck both extend Vehicle, inheriting shared fields like licensePlate.
- **Polymorphism** — the same call producing different behavior depending on the actual runtime type. Calling vehicle->getSpotSize() returns a different enum for a Car vs. a Motorcycle vs. a Truck, without the caller needing an if/else on type.

encapsulation abstraction inheritance polymorphism

1.2 BASIC Composition vs inheritance — when do you prefer which?

Inheritance models an "is-a" relationship and is appropriate when subtypes genuinely share behavior that shouldn't vary independently — a Car *is a* Vehicle. **Composition** models a "has-a" relationship — an object is built from other objects it delegates to — and is preferred whenever behavior needs to vary independently of type, or when deep inheritance hierarchies would become rigid. Search sources are explicit that **overusing inheritance is one of the most frequently cited LLD mistakes**: a Bird class with a fly() method breaks the moment a Penguin subclass is added (this is also a direct Liskov Substitution violation, 1.4), whereas composing a FlyBehavior object into each bird (the classic Strategy pattern setup, Section 2) lets a Penguin simply be composed with a CannotFly behavior instead. The rule of thumb repeated across sources: "favor composition over inheritance" by default, and reach for inheritance only when the hierarchy is genuinely stable and shallow.

is-a vs has-a

favor composition over inheritance

1.3 BASIC State the five SOLID principles.

LETTER	PRINCIPLE	ONE-LINE MEANING
S	Single Responsibility	A class should have only one reason to change
O	Open/Closed	Open for extension, closed for modification
L	Liskov Substitution	A subtype must be usable anywhere its base type is expected, without surprises
I	Interface Segregation	Don't force a class to depend on methods it doesn't use
D	Dependency Inversion	Depend on abstractions, not concrete implementations

SRP

OCP

LSP

ISP

DIP

1.4

INTERMEDIATE

Walk through SRP and OCP with a concrete class, a violation, and a fix.

```
// SRP VIOLATION: one class does parking allocation, payment, AND ticket printing
class ParkingSpot {
public:
    void assignVehicle(Vehicle* v);
    void processPayment(double amount);    // unrelated reason to change
    void printTicket();                    // another unrelated reason to change
};

// FIX: split by reason-to-change into ParkingSpot, PaymentService, TicketPrinter
// each class now has exactly one reason to change
```

This exact combination — "payment logic, parking allocation, and ticket generation inside one class" — is named directly in search sources as the canonical SRP-violation example interviewers use. For **OCP**: a `PaymentService` that switches on payment type with an `if/else` chain must be *modified* every time a new payment method is added; defining a `PaymentStrategy` interface (Section 2.1) that concrete strategies implement means adding UPI support is a pure *extension* — a new class — with zero changes to existing, already-tested code.

reason to change

extension without modification

REAL INTERVIEW SIGNAL

Sources state directly: "what scores well is mapping each principle to a specific class on your diagram with a concrete follow-up scenario — name the class, name the change that would violate the principle, name the fix." This Q&A format is deliberately structured to rehearse exactly that.

Liskov Substitution — the classic violation, and why it matters for LLD specifically.

```
class Rectangle {
public:
    virtual void setWidth(int w) { width = w; }
    virtual void setHeight(int h) { height = h; }
    int area() const { return width * height; }
protected:
    int width, height;
};

class Square : public Rectangle {
public:
    void setWidth(int w) override { width = height = w; } // silently changes height too!
    void setHeight(int h) override { width = height = h; }
};

// Code written correctly against Rectangle now silently breaks when given a Square:
void resize(Rectangle& r) { r.setWidth(5); r.setHeight(4); /* expects area()==20 */ }
// resize(square) gives area() == 16, not 20 - LSP violated
```

Square is a valid mathematical is-a relationship with Rectangle, but the *behavioral contract* callers rely on (setting width doesn't affect height) is silently broken — this is precisely why LSP is about behavioral substitutability, not just type-hierarchy correctness. In LLD interviews this matters concretely whenever a design uses inheritance for two entities that seem related but have different invariants — e.g. a MotorcycleSpot that unexpectedly rejects an operation every other ParkingSpot subtype allows is an LSP red flag the interviewer is listening for you to catch.

behavioral substitutability

Square-Rectangle problem

1.6

ADVANCED

ISP and DIP together — how do they enable testability?

```
// ISP VIOLATION: a fat interface forces every implementer to stub methods it doesn't need
class IVehicle {
virtual void drive() = 0;
virtual void fly() = 0;    // a Car has no business implementing this
};

// FIX: split into focused interfaces
class IDrivable { virtual void drive() = 0; };
class IFlyable { virtual void fly() = 0; };

// DIP: depend on the abstraction, not a concrete class, so tests can substitute a fake
class ParkingSpotAllocator {
public:
    explicit ParkingSpotAllocator(std::shared_ptr<IPricingStrategy> pricing)
        : pricing_(std::move(pricing)) {} // injected, not "new SurgePricing()" hardcoded inside
private:
    std::shared_ptr<IPricingStrategy> pricing_;
};
```

ISP keeps interfaces narrow so a class only depends on methods it actually uses — reducing unnecessary coupling and unnecessary recompilation/retesting blast radius when an unrelated method changes. DIP — depending on `IPricingStrategy` rather than a concrete `SurgePricing` class, and receiving it via constructor injection rather than constructing it internally — is precisely what makes unit testing tractable (Section 6.6 in the framework): a test can inject a `FakePricingStrategy` that returns a fixed value, isolating the class under test from real pricing logic entirely. Search sources note this directly: "DIP facilitates dependency injection, allowing easy swapping of real dependencies with test doubles during unit testing" — this is the concrete, practical payoff interviewers want named, not just the abstract principle.

fat interface

constructor injection

test doubles

DRY, KISS, and YAGNI — three simple rules that come up constantly, explained in one sentence each.

- **DRY** (Don't Repeat Yourself) — if you find yourself copy-pasting the same logic in two places, pull it into one shared function/class instead, so a future fix only needs to happen once.
- **KISS** (Keep It Simple, Stupid) — build the simplest design that actually satisfies the stated requirements; a clever, complicated solution that does more than asked is not a bonus, it's a liability someone else has to understand later.
- **YAGNI** (You Aren't Gonna Need It) — don't build a feature or a layer of flexibility "just in case" it's needed someday; build it when a real requirement actually asks for it, not before.

These three are the beginner-level, practical companions to SOLID above — where SOLID is mostly about how to structure classes correctly, DRY/KISS/YAGNI are about not over-building in the first place. In an interview, all three show up as a specific kind of feedback: if you're adding a configuration system, an abstract base class, or a caching layer that the stated requirements never asked for, a good interviewer will ask "why did you add this?" — and "I thought we might need it later" is precisely the YAGNI violation this principle warns against (and directly overlaps with Section 11's "over-engineering" red flag).

DRY KISS YAGNI avoid premature generalization

WHAT THE WEB SOURCES EMPHASIZE

Adaface, Indeed, and CalibreOS's SOLID interview guides all converge on the same evaluation criterion: naming a specific class, a specific change that would violate the principle, and a specific fix — never abstract definitions alone. Separately, current sources on DRY/KISS/YAGNI describe them explicitly as "guides, not rigid laws" — meant to be applied with judgment, not mechanically. This shaped every question above to end in a code-level violation-and-fix, not a one-line definition.

02 Design Patterns for LLD

One source is unusually precise here: **Strategy, State, Observer, and Factory cover roughly 90% of LLD problems** at Amazon, Flipkart, Swiggy, and Google — so this section leads with those four in depth. It then covers the full Gang-of-Four syllabus interviewers draw the remaining 10% from, organized by the three standard categories (Creational, Structural, Behavioral), so nothing commonly asked is missing.

2.1 BASIC Strategy pattern — why is it "the most common pattern in LLD interviews"?

```
class IPricingStrategy {
public:
    virtual double computeFee(int hours) const = 0;
    virtual ~IPricingStrategy() = default;
};
class HourlyPricing : public IPricingStrategy {
public:
    double computeFee(int hours) const override { return hours * 2.0; }
};
class FlatRatePricing : public IPricingStrategy {
public:
    double computeFee(int hours) const override { return 10.0; }
};
```

Strategy encapsulates an *algorithm* or *behavior* behind a common interface, letting it be swapped at runtime without touching the class that uses it. It's the most-cited pattern precisely because it directly tests whether a candidate can isolate behavior that varies — replacing an `if/else-on-type` chain with interchangeable, independently-testable objects — and it's simultaneously the concrete mechanism for satisfying OCP (1.4). Almost every case study in this guide uses it at least once (pricing in Parking Lot, eviction policy in LRU Cache, throttling algorithm in Rate Limiter).

Strategy

interchangeable behavior

2.2 BASIC Factory pattern — what problem does it solve?

```
class VehicleFactory {
public:
    static std::unique_ptr<Vehicle> create(VehicleType type) {
        switch (type) {
            case VehicleType::Car:         return std::make_unique<Car>();
            case VehicleType::Motorcycle: return std::make_unique<Motorcycle>();
            case VehicleType::Truck:        return std::make_unique<Truck>();
        }
        throw std::invalid_argument("unknown vehicle type");
    }
};
```

Factory centralizes object creation behind one method instead of scattering `new Car()/new Truck()` calls throughout the codebase — per search sources, this is exactly the practical framing interviewers want: "instead of `new EmailNotification()` scattered throughout your code, you call `notificationFactory.create(type)`." The payoff is that adding a new vehicle type touches exactly one place, and callers never need to know concrete class names at all — they depend only on the abstract `Vehicle` interface (DIP, 1.6).

Factory

centralized creation

2.3

INTERMEDIATE

Observer pattern — when does a design "need" it?

```
class ISpotObserver {
public:
    virtual void onSpotFreed(int spotId) = 0;
};
class DisplayBoard : public ISpotObserver {
public:
    void onSpotFreed(int spotId) override { /* update the "spots available" sign */ }
};
class ParkingFloor {
public:
    void addObserver(ISpotObserver* obs) { observers_.push_back(obs); }
    void freeSpot(int id) {
        /* ...mark free... */
        for (auto* obs : observers_) obs->onSpotFreed(id); // notify every subscriber
    }
private:
    std::vector<ISpotObserver*> observers_;
};
```

Observer is the right call whenever **multiple, independent components must react to one state change** without the state-owning class needing to know who they are: a spot being freed might need to update a display board, notify a waiting-customer queue, and update an analytics counter — the `ParkingFloor` shouldn't hardcode calls to all three; it just notifies its list of registered observers. Search sources describe the tell precisely: "a stock price update that triggers multiple displays, an order placement that notifies inventory, analytics, and notifications — you're looking at Observer."

Observer

publish/subscribe

2.4 INTERMEDIATE State pattern — how do you know your design needs a state machine?

Per search sources: "state machines are less common than Strategy or Observer, but when you need one, it's usually the centerpiece of your entire design... if there's a state machine in your solution, the interview is probably organized around it." The tell: an entity has a small, fixed set of named states, each state permits only a specific subset of transitions/operations, and behavior for the "same" method genuinely differs by state — an Elevator that's Idle vs Moving vs DoorsOpen responds completely differently to a new floor request. The State pattern encodes each state as its own class implementing a common interface, so invalid transitions (accepting a new destination floor while doors are open) become structurally hard to express by accident, rather than relying on scattered `if (state == MOVING)` guards throughout the codebase. **Drawing the state diagram is explicitly named as one of the best ways to communicate this design in the interview** — see the Elevator case study (Section 7) for a full worked example.

State pattern

state diagram

2.5 ADVANCED

Beyond the top four — Singleton, Decorator, Command: when are they actually justified vs. over-engineering?

PATTERN	GENUINELY JUSTIFIED WHEN...	OVER-ENGINEERING RED FLAG
Singleton	Exactly one coordinating instance must exist system-wide (a ParkingLot itself, a logging registry) and this is a real constraint, not a convenience	Used as a substitute for proper dependency injection just to avoid passing a reference around
Decorator	Behavior needs to be layered/composed dynamically (adding "premium support" wrapping onto a base notification) without an explosion of subclasses	Used where a simple boolean flag or Strategy would communicate the same intent more plainly
Command	Operations need to be queued, logged, undone, or scheduled as first-class objects (an elevator's pending floor requests)	Wrapping every simple method call as a "Command" object with no actual queuing/undo need

This table exists specifically because search sources repeatedly flag over-engineering — adding patterns that aren't earning their complexity — as a top failure mode (Section 11). The senior-level signal is knowing a pattern exists *and* correctly deciding not to use it for a given problem, not maximizing pattern count per design.

Singleton

Decorator

Command

pattern restraint

2.6 INTERMEDIATE Builder pattern — when is a constructor not enough?

```
class Pizza {
public:
    class Builder {
    public:
        Builder& setSize(int s)          { size_ = s; return *this; }
        Builder& addTopping(std::string t) { toppings_.push_back(std::move(t)); return *this; }
        Pizza build()                    { return Pizza(size_, toppings_); }
    private:
        int size_ = 12;
        std::vector<std::string> toppings_;
    };
private:
    Pizza(int size, std::vector<std::string> toppings) : size_(size), toppings_(std::move(toppings)) {}
    int size_; std::vector<std::string> toppings_;
};

// Pizza p = Pizza::Builder().setSize(14).addTopping("cheese").addTopping("olives").build();
```

Builder solves the problem of constructing an object that has **many optional parameters, some of which depend on others, or where the construction itself has several steps** — a plain constructor with 8 parameters (several boolean/optional) is unreadable at the call site and error-prone (easy to swap two same-typed arguments by position). Builder makes each step a named, chainable call, and defers actually assembling the final immutable object to one `build()` call. It's a very common, direct follow-up to any case study involving a complex value object — "what if Expense had 10 optional fields?" (Splitwise) is a natural Builder prompt.

Builder

fluent construction

telescoping constructor problem

Structural patterns

2.7

INTERMEDIATE

Adapter vs. Facade — both "simplify," but solve different problems.

```
// Adapter: makes an EXISTING, incompatible interface match the one your code expects
class ILegacyPaymentGateway { public: virtual void doPay(std::string amountStr) = 0; };
class PaymentAdapter : public IPaymentProcessor { // IPaymentProcessor::charge(double) is what your code
calls
public:
    explicit PaymentAdapter(std::shared_ptr<ILegacyPaymentGateway> legacy) : legacy_(legacy) {}
    void charge(double amount) override { legacy_->doPay(std::to_string(amount)); }
private:
    std::shared_ptr<ILegacyPaymentGateway> legacy_;
};

// Facade: hides a whole SUBSYSTEM of several classes behind one simple entry point
class CheckoutFacade {
public:
    void checkout(Cart& cart) {
        inventory_.reserve(cart); // three separate subsystems, coordinated in one call
        payment_.charge(cart.total());
        shipping_.schedule(cart);
    }
private:
    InventoryService inventory_; PaymentService payment_; ShippingService shipping_;
};
```

Adapter is about compatibility — one existing class's interface doesn't match what your code needs, so you wrap it. **Facade** is about simplicity — several real classes genuinely need to collaborate, and you give callers one clean entry point instead of forcing them to orchestrate all three. A quick interview tell: if you're wrapping *one* misshapen interface, it's Adapter; if you're hiding the coordination of *several* classes behind one method, it's Facade.

Adapter

Facade

interface compatibility vs. subsystem simplification

2.8 ADVANCED

Decorator vs. Proxy — both wrap an object behind the same interface, so what's different?

```
class INotifier { public: virtual void send(std::string msg) = 0; };

// Decorator: ADDS behavior, and decorators can be stacked arbitrarily
class UrgentDecorator : public INotifier {
public:
    explicit UrgentDecorator(std::shared_ptr<INotifier> inner) : inner_(inner) {}
    void send(std::string msg) override { inner_->send("[URGENT] " + msg); }
private:
    std::shared_ptr<INotifier> inner_;
};

// Proxy: CONTROLS access to the same operation — same interface, no new behavior added
class RateLimitedNotifierProxy : public INotifier {
public:
    RateLimitedNotifierProxy(std::shared_ptr<INotifier> real, RateLimiter& rl) : real_(real), limiter_(rl) {}
    void send(std::string msg) override {
        if (limiter_.allowRequest("notifier")) real_->send(msg); // same call, gated
    }
private:
    std::shared_ptr<INotifier> real_; RateLimiter& limiter_;
};
```

Both compile to "a class implementing the same interface, holding a reference to another instance of it." The distinguishing intent: **Decorator** exists to layer new, additive behavior on top of the wrapped object — you can stack several decorators, and the caller gets more than the base object did. **Proxy** exists to control *access* to the wrapped object — lazy loading (don't construct the real, expensive object until first use), access control (permission checks), or rate limiting (exactly the Section 9 case study, reused here) — the caller gets the *same* behavior, just gated or deferred. Interviewers use this pair specifically to test whether "wraps the same interface" is being conflated with "does the same thing" — it deliberately isn't.

Decorator

Proxy

additive vs. gating wrapper

Behavioral patterns beyond Strategy, State, Observer

```
// Template Method: fixes the ALGORITHM'S SKELETON, Lets subclasses fill in specific steps
class DataExporter {
public:
    void exportData() {          // the skeleton – not virtual, never overridden
        fetchData();
        formatData();           // the one step that varies – subclasses override just this
        writeToDestination();
    }
protected:
    virtual void formatData() = 0;
private:
    void fetchData() { /* shared, identical for every subclass */ }
    void writeToDestination() { /* shared */ }
};
// class CsvExporter : public DataExporter { void formatData() override { /* CSV-specific */ } };

// Chain of Responsibility: a request passes along a chain until SOMETHING handles it
class SupportHandler {
public:
    void setNext(std::shared_ptr<SupportHandler> next) { next_ = next; }
    virtual void handle(Ticket& t) {
        if (canHandle(t)) { resolve(t); return; }
        if (next_) next_>handle(t); // pass along; this handler doesn't know who's next
    }
protected:
    virtual bool canHandle(const Ticket&) = 0;
    virtual void resolve(Ticket&) = 0;
private:
    std::shared_ptr<SupportHandler> next_;
};
```

Template Method is about one fixed sequence of steps where only *some* steps vary by subtype — the caller always gets the same overall algorithm shape. **Chain of Responsibility** is about routing a single request through an open-ended sequence of candidate handlers until one accepts it — used constantly in real systems as the mental model behind middleware/interceptor chains (an HTTP request passing through auth → logging → rate-limiting middleware, each free to handle it or pass it on). A support-ticket escalation system (tier-1 → tier-2 → tier-3) is the canonical LLD prompt for this pattern specifically.

Template Method

Chain of Responsibility

middleware chain

Full Gang-of-Four reference (for completeness)

The four deep-dives above (2.1–2.4) plus the six just covered (2.6–2.9) are what search sources confirm as actually recurring. The remaining classic GoF patterns round out the syllabus for

completeness — asked less often standalone, but worth recognizing by name and one-line intent if an interviewer name-drops one:

PATTERN	CATEGORY	ONE-LINE INTENT
Singleton	Creational	Exactly one instance, globally accessible (2.5)
Factory Method	Creational	Centralize creation of one product type behind a method (2.2)
Abstract Factory	Creational	Create <i>families</i> of related objects together (e.g. a UI toolkit's matching Button+Checkbox+Menu per theme)
Builder	Creational	Step-by-step construction of a complex object (2.6)
Prototype	Creational	Create new objects by cloning an existing, pre-configured instance rather than constructing from scratch
Adapter	Structural	Make one incompatible interface match what callers expect (2.7)
Facade	Structural	One simple entry point over a multi-class subsystem (2.7)
Decorator	Structural	Add behavior dynamically by wrapping, stackable (2.5, 2.8)
Proxy	Structural	Control/gate access to an object behind the same interface (2.8)
Composite	Structural	Treat a single object and a group of objects through the same interface (a file and a folder both expose <code>getSize()</code>)
Bridge	Structural	Decouple an abstraction from its implementation so both can vary independently (a Shape hierarchy independent of its Renderer hierarchy)
Flyweight	Structural	Share common state across many objects to reduce memory (glyph objects in a text editor sharing font-data)
Strategy	Behavioral	Swappable algorithm behind a common interface (2.1)
State	Behavioral	Behavior varies by an entity's current named state (2.4)
Observer	Behavioral	Many subscribers react to one subject's change (2.3)
Template Method	Behavioral	Fixed algorithm skeleton, overridable individual steps (2.9)
Chain of Responsibility	Behavioral	Pass a request along handlers until one accepts it (2.9)

PATTERN	CATEGORY	ONE-LINE INTENT
Command	Behavioral	Encapsulate a request as an object, enabling queuing/undo/logging (2.5)
Iterator	Behavioral	Traverse a collection without exposing its internal structure
Mediator	Behavioral	Centralize how a set of objects communicate, instead of all-to-all references (an air-traffic-control tower coordinating planes)
Memento	Behavioral	Capture and restore an object's internal state without violating encapsulation (undo/redo)
Visitor	Behavioral	Add a new operation across a class hierarchy without modifying the classes themselves

Decision guide: "I have this problem — which pattern/principle do I reach for?"

This is the quick-reference version of everything above — read it right to left: start from the symptom you're actually facing, not from a pattern name you're trying to justify using.

IF YOU'RE THINKING...	REACH FOR	NOT THIS (COMMON OVER-REACH)
"This class does three unrelated things"	SRP — split into three classes (1.4)	A Facade that just hides the mess instead of fixing it
"I need a new payment type without touching old code"	Strategy + OCP (1.4, 2.1)	Adding another if/else branch
"This entity behaves completely differently depending on its status"	State (2.4)	A pile of boolean flags checked everywhere
"Several unrelated things need to react when X happens"	Observer (2.3)	The subject class calling each of them directly by name
"Creating this object is genuinely complicated or type-dependent"	Factory (2.2)	new scattered across the whole codebase
"This object has 8 optional constructor parameters"	Builder (2.6)	Several overloaded constructors nobody can remember the order of
"I need to make this old/incompatible class fit my new interface"	Adapter (2.7)	Rewriting the old class from scratch
"Callers shouldn't have to coordinate three services themselves"	Facade (2.7)	Making every caller learn all three services' APIs
"I want to add behavior without touching the original class, and it should stack"	Decorator (2.8)	Subclassing for every combination of extra behavior
"I need to gate/delay access to the real object"	Proxy (2.8)	Scattering the access-check logic into every caller
"I'm not sure any pattern is needed yet"	YAGNI — build the plain version first (1.7)	Adding a pattern "for flexibility" with no concrete need yet

This table is the single page to re-read the morning of your interview if you only have five minutes — it's a condensed version of everything Sections 1–2 explain in full.

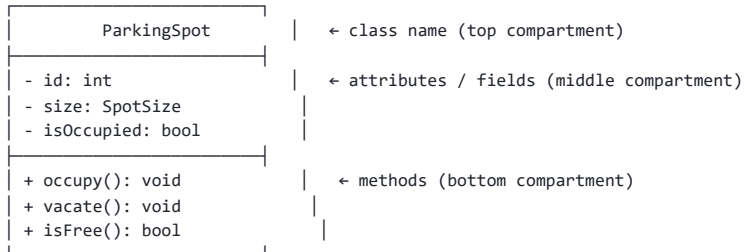
WHAT THE WEB SOURCES EMPHASIZE

The specific claim "Strategy, State, Observer, and Factory cover ~90% of LLD interviews at Amazon, Flipkart, Swiggy, and Google" appears explicitly in search results (Grind Engineer) — this section leads with those four in depth, and then completes the syllabus (Builder, Adapter, Facade, Decorator, Proxy, Template Method, Chain of Responsibility, plus a full reference table for the remaining classic GoF set) so every commonly named pattern is covered, not just the top four.

03 UML & Class Modeling

Sources agree the diagram itself is evaluated — but also agree that simplified notation is expected and welcomed, not full formal UML. Starts with the basic box notation for readers who've never drawn one before.

3.0 BASIC What does a single class actually look like drawn as a box?



Every UML class diagram box has exactly these **three compartments**, top to bottom: the class name, its attributes (data it holds — matching the "attribute" idea from 1.0), and its methods (actions it can perform). A `-` prefix on a member means **private** (only this class can access it directly — the encapsulation idea from 1.1); a `+` prefix means **public** (any other class can call it). Once you can read one box, reading a full diagram is just reading several boxes and the lines connecting them (3.1) — there is no more notation to memorize beyond this.

class box (3 compartments)

+ public / - private

3.1 **BASIC** What relationships does a class diagram need to express, and how?

RELATIONSHIP	MEANING	NOTATION (ARROW)
Association	"Uses" — a general connection between two classes	plain line
Aggregation	"Has-a," but the part can outlive the whole	line with a hollow diamond
Composition	"Has-a," and the part's lifetime is owned by the whole	line with a filled diamond
Inheritance	"Is-a"	line with a hollow triangle, pointing at the parent
Realization	Implements an interface	dashed line with a hollow triangle

The distinction between aggregation and composition matters concretely: a `ParkingFloor` *composes* `ParkingSpots` (a spot has no meaning outside its floor — destroy the floor, its spots go with it), but a `ParkingLot` merely *aggregates* `Vehicles` (a car still exists after it leaves the lot).

association

aggregation

composition

realization

3.2 **INTERMEDIATE** How much diagram formality does an interview actually expect?

Per Hello Interview's own framework: "ask whether simplified class notation is acceptable — it's faster to write and easier to discuss, and in most cases, it will be." The practical standard used throughout this guide's case studies: boxes with a class name, key fields, and key method signatures, connected by labeled arrows for the relationships in 3.1 — full formal UML (visibility markers, multiplicity on every association, stereotypes) is rarely what's being graded. What *is* graded: whether the relationships are directionally correct (who owns whom, who depends on whom) and whether the diagram matches the responsibilities you narrate out loud — a diagram that contradicts your own verbal description is a worse signal than a rougher but internally consistent one.

simplified class notation

3.3 ADVANCED

How do you decide what belongs on a class vs. what belongs on a separate "manager/service" class?

The recurring, SRP-driven heuristic (1.4): a class should own the state and behavior that's *intrinsic* to a single instance of that concept — a `ParkingSpot` knows its own occupied/free status and size; it should not know how to compute a bill for the whole lot. Cross-cutting coordination that spans *many* instances or requires global knowledge (assigning the nearest free spot across an entire floor, computing a fee that depends on a pricing policy) belongs on a separate coordinating class (a `ParkingLot` or `SpotAllocator`) that *holds references to* the entity classes rather than being one. This is exactly the split search sources point to when they criticize combining "payment logic, parking allocation, and ticket generation inside one class" — each of those is really a distinct cross-cutting concern deserving its own coordinator, not a natural property of any single entity.

entity vs. coordinator class

WHAT THE WEB SOURCES EMPHASIZE

Hello Interview's delivery framework explicitly tells candidates to ask upfront whether simplified notation is fine — a detail this section leads with directly, since over-formalizing UML under time pressure is a documented, avoidable time sink.

04 Concurrency in LLD

Increasingly common as a follow-up layer on top of a correct single-threaded design, per multiple sources on LRU Cache and Rate Limiter specifically — thread safety is rarely the opening ask, but is a near-guaranteed deep-dive.

4.1 BASIC

Why does a correct single-threaded LLD design often break under concurrency?

A design that's logically correct for one caller at a time can still have **race conditions** — two threads interleaving reads and writes to shared mutable state such that the combined result is wrong, even though neither thread's own logic has a bug. The canonical LLD example: two threads both call `parkVehicle()` on the same `ParkingLot`, both read "spot 5 is free," and both assign spot 5 to a different car — the check-then-act sequence (check free, then mark occupied) isn't atomic unless explicitly made so. This is precisely why a design's data structures (a shared `std::vector<ParkingSpot>`, a shared cache map) need explicit synchronization the moment more than one thread can call into them concurrently — something not required, and not free, in the single-threaded version of the same design.

race condition

check-then-act

4.2 INTERMEDIATE Coarse-grained locking vs fine-grained locking — the trade-off.

Coarse-grained locking — one mutex guarding the entire structure (a single `std::mutex` around all of `ParkingLot`) — is simple to reason about and hard to get wrong, at the cost of serializing every operation, even ones that don't actually conflict (parking a car on floor 1 blocks a totally unrelated lookup on floor 3). **Fine-grained locking** — a separate lock per floor, or per spot, or (per search sources on LRU Cache) using a concurrent map plus a concurrent deque so reads can proceed lock-free while only structural mutations synchronize — allows real parallelism for genuinely independent operations, at the cost of real complexity: multiple locks raise the risk of deadlock if they're ever acquired in inconsistent order across different code paths. The interview-grade answer names this trade-off explicitly and picks coarse-grained as the default, safe starting point, escalating to fine-grained only when asked to optimize for throughput under contention (a standard LRU Cache/Rate Limiter follow-up, per Sections 8 and 9).

coarse-grained lock

fine-grained lock

deadlock risk

4.3 ADVANCED What does "thread safety without locks" mean, and when is it worth it?

Per search sources on high-performance LRU Cache implementations: using a **concurrent hash map** (lock-free or fine-grained-locked internally) for the key lookup, combined with a **concurrent doubly-linked structure** for recency ordering, lets multiple threads perform `get()` reads with minimal contention, only synchronizing narrowly around the specific structural mutation (moving a node to the front) rather than locking the entire cache for every access. This is a genuine performance win under high read concurrency, but it's also real added complexity (atomic operations, careful memory ordering, correctness under torn writes) that is *not* worth introducing unless the interviewer has specifically pushed into a high-throughput, high-contention follow-up — reaching for lock-free structures unprompted, on a first pass, is itself a documented instance of the over-engineering red flag from Section 11.

concurrent hash map

lock-free reads

WHAT THE WEB SOURCES EMPHASIZE

Search sources on both LRU Cache and Rate Limiter frame concurrency identically: "thread safety involves proper locking with correct granularity, not just a lock around everything" — the explicit expectation is a graduated answer (start coarse, justify any move to fine-grained), which is exactly how 4.2 and 4.3 are structured.

05 How to Solve Any LLD Problem — A Repeatable Framework

Synthesized directly from Hello Interview's delivery framework, GeeksforGeeks' preparation guide, and the recurring structure across dozens of aggregated interview reports. Every case study in Sections 6–10 follows exactly these nine steps, in order.

Step 1 — Requirements clarification (spend the first 5–8 minutes here, always)

LLD prompts are deliberately fuzzy and under-specified, mirroring real product requirements. Sources are explicit: never assume — ask. Work down these themes:

- **Primary capabilities** — what operations must the system support, at minimum?
- **Scope boundaries** — what's explicitly out of scope for this session? (e.g. "assume payment processing is a black box — I won't ask you to implement Stripe")
- **Scale** — one building or a chain of 500 parking lots? A rough scale changes whether in-memory data structures are even reasonable.

- **Must-haves vs. nice-to-haves** — confirm explicitly which features are core to the session vs. which you'll mention as future extensions.
-

Step 2 — State assumptions and constraints out loud

Once ambiguity remains (it always will, on some axis), state the assumption you're proceeding with rather than silently picking one — "I'll assume a single parking lot, single-threaded for now, and revisit concurrency if we have time" is itself a strong signal of structured thinking.

Step 3 — Use cases and non-functional requirements

List the functional use cases as short actor-action statements ("a driver parks a vehicle," "an operator issues a refund"). Even a minimal non-functional pass matters: is fast lookup more important than fast insert? Is this read-heavy or write-heavy? Does it need to survive a process restart (persistence) or is in-memory acceptable? This directly drives Step 5's data-structure choice.

Step 4 — Identify entities, responsibilities, and collaborators

Extract nouns from the use cases as candidate classes, then immediately apply SRP (1.4) — merge or split until each class has exactly one reason to change. For each entity, ask "who needs to talk to whom?" — this becomes your class diagram's arrows before you've written a single method signature.

Step 5 — Interfaces, abstractions, and pattern selection

Wherever a use case says "and this might vary" (pricing scheme, eviction policy, notification channel), reach for Strategy (2.1) behind an interface rather than an `if/else`. Wherever an entity has named states with different legal operations per state, reach for State (2.4). Apply Factory (2.2) only where object creation logic is genuinely non-trivial or varies by input — not reflexively on every class.

Step 6 — Data structures

Choose concretely, and justify with a complexity note: a hash map for $O(1)$ ID lookup, a min-heap for "next available/soonest" queries, a doubly-linked list paired with a hash map for LRU-style recency ordering (Section 8). Naming the Big-O of your core operations unprompted is a strong, cheap signal.

Step 7 — Error handling and edge cases

Per the mistakes research (Section 11), this is one of the most commonly *skipped* steps, and one of the most differentiating when done well. For every public method, ask: what happens on invalid input, on a not-found lookup, on a capacity-exceeded condition, on a duplicate operation? Decide explicitly whether to throw, return an optional/sentinel, or return a result/error type — and say why.

Step 8 — Concurrency pass (only if relevant, or if asked)

Apply Section 4's framework: default to coarse-grained locking, name where a race condition would occur without it, and only escalate to fine-grained/lock-free structures if the interviewer pushes for throughput under contention.

Step 9 — Test strategy and complexity/maintainability notes

Name what you'd unit test (each class in isolation, using DIP-injected fakes for collaborators — 1.6), what you'd integration test (the full flow across several real classes), and what's worth mocking (anything with external side effects — payment, notifications, the system clock in a time-dependent design like Rate Limiter). Close by naming the design's actual extension points — this is frequently the last two minutes of the interview and directly sets up Section 10's follow-up questions.

WHAT INTERVIEWERS LISTEN FOR

A candidate who narrates *why* at every step — "I'm using Strategy here because pricing is explicitly called out as something that varies by vehicle type and time of day" — scores higher than one who arrives at an identical diagram silently. Multiple sources state this almost verbatim: "interviewers can't read your mind."

06 Case Study: Parking Lot

FREQUENCY SIGNAL: NAMED DIRECTLY AS "SHOULD BE THE FIRST QUESTION IN YOUR LLD LIST" AND REPORTED IN ~40% OF SDE-1 ROUNDS PER GFG'S AGGREGATED DATA; A REAL REPORTED PROMPT AT WAYFAIR (SDE-II)

Problem statement, as asked

"Design an object-oriented parking lot system that supports multiple vehicle types (motorcycle, car, truck/XL) and multiple spot sizes, tracks availability, and issues a ticket on entry and computes a fee on exit." (Wayfair's reported phrasing specifically calls out XL/L/M/S sized spots.)

Clarifying questions to ask: Single lot or multiple floors/lots? Can a smaller vehicle park in a larger spot (and should it, if no exact match is free)? Is pricing flat, hourly, or dynamic? Is payment processing in scope, or a black box? Do we need to support reservations, or only walk-in/first-come availability?

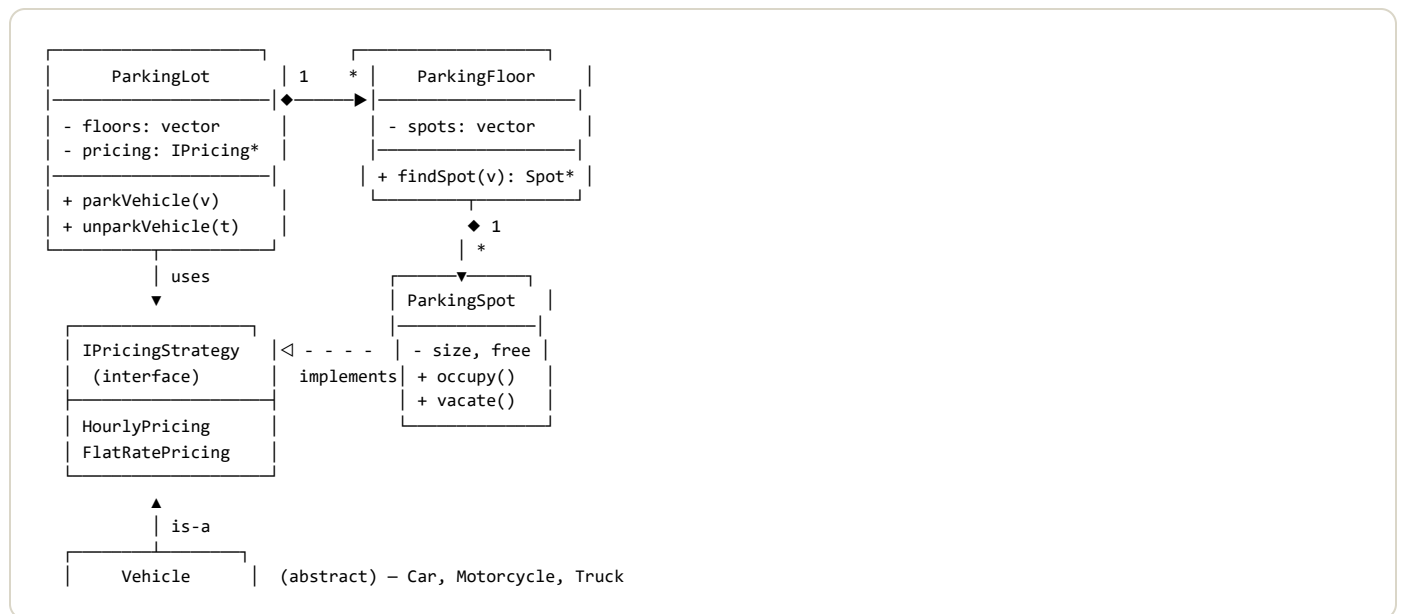
Functional & non-functional requirements

- **FR1** — Park a vehicle: find and assign the smallest available spot that fits the vehicle, issue a ticket with entry time.
- **FR2** — Unpark a vehicle: given a ticket, compute the fee from entry/exit time and pricing strategy, free the spot.
- **FR3** — Query real-time availability, by spot size and overall.
- **FR4** — Support multiple floors, each with its own spots.
- **NFR1** — Assignment must be fast (near $O(1)/O(\log n)$) even with thousands of spots — this is read-heavy on availability checks.
- **NFR2** — Extensible to new vehicle types and new pricing schemes without modifying existing classes (OCP, 1.4).

Core classes & responsibilities

CLASS	RESPONSIBILITY
Vehicle (abstract) / Car, Motorcycle, Truck	Knows its own size category; polymorphic (1.1) so allocation code never branches on concrete type
ParkingSpot	Owns its own occupied/free state and size; exposes occupy()/vacate() (encapsulation, 1.1)
ParkingFloor	Composes a set of ParkingSpots (composition, 3.1); finds the best-fit free spot on this floor
ParkingLot	Composes floors; top-level coordinator for parkVehicle/unparkVehicle (Singleton — exactly one lot instance coordinates all floors, 2.5)
Ticket	Immutable record of vehicle, spot, entry time
IPricingStrategy / HourlyPricing, FlatRatePricing	Strategy (2.1) — fee computation, swappable without touching ParkingLot

Class diagram (simplified notation, per 3.2)



Key interaction: parkVehicle(vehicle)

1. **ParkingLot** iterates its floors (nearest-first, or floor 0 first) and asks each **ParkingFloor::findSpot(vehicle)**.
2. **ParkingFloor** scans its spot collection for the smallest free spot $\geq$ the vehicle's required size (best-fit, not first-fit, to leave large spots for large vehicles).
3. If found: `spot.occupy(vehicle)`; **ParkingLot** creates a **Ticket** with entry timestamp and returns it. If not found on any floor: throw/return a "lot full" result (Step 7 of the framework).

Data model / in-memory representation

```
// Spots grouped by size in each floor for O(1) "any free spot of size X" lookup
std::unordered_map<SpotSize, std::vector<ParkingSpot*>> spotsBySize_;
// A min-heap or simple free-list per size avoids a full scan on each park() call
```

Public API (key signatures)

```
class ParkingLot {
public:
    std::optional<Ticket> parkVehicle(std::shared_ptr<Vehicle> vehicle);
    double unparkVehicle(const Ticket& ticket);
    int availableSpots(SpotSize size) const;
};
```

Design choices, trade-offs & extension scenarios

- **Best-fit vs. first-fit allocation** — best-fit (smallest spot that still fits) preserves large spots for vehicles that actually need them, at the cost of a slightly more complex lookup (grouping by size, as above) versus a naive first-free scan.
- **Extension: add electric vehicle charging spots.** Add a `ChargingSpot` : `ParkingSpot` subtype and an `IElectricVehicle` marker interface — no change to `ParkingFloor::findSpot`'s core logic if it's written against the `ParkingSpot` base (OCP holding up under a real follow-up).
- **Extension: reservations.** Requires a new `Reservation` concept that temporarily holds a spot without physical occupancy — a good discussion point for why this is a distinct state, not just reusing `occupy()`.

Concurrency considerations

Per Section 4: two cars arriving simultaneously must not both be assigned the same spot. Coarse-grained default: a single mutex around `ParkingFloor::findSpot + occupy()`, treated as one atomic critical section, since "find a free spot" and "mark it occupied" must happen together (4.1's check-then-act problem). A natural fine-grained escalation: one lock per floor rather than one lock for the whole lot, since two cars entering on different floors don't actually conflict.

Test cases & what to mock

- Park a vehicle when spots are available → returns a valid ticket, spot marked occupied.
- Park a vehicle when the lot is full → returns empty/throws, no state mutated.
- Unpark with a valid ticket → correct fee computed (inject a **fake IPricingStrategy** returning a fixed rate, and a **fake clock** for deterministic entry/exit timestamps — both are the "external dependency" boundaries worth mocking per the framework's Step 9).

- Concurrent park calls for the last remaining spot → exactly one succeeds (this is the test that actually exercises Section 4's locking).

Minimal C++ code skeleton

```

enum class SpotSize { Motorcycle, Compact, Large };

class Vehicle {
public:
    virtual SpotSize requiredSize() const = 0;
    virtual ~Vehicle() = default;
};

class Car : public Vehicle {
public: SpotSize requiredSize() const override { return SpotSize::Compact; }
};

class ParkingSpot {
public:
    ParkingSpot(int id, SpotSize size) : id_(id), size_(size) {}
    bool isFree() const { return free_; }
    SpotSize size() const { return size_; }
    void occupy() { free_ = false; }
    void vacate() { free_ = true; }
private:
    int id_; SpotSize size_; bool free_ = true;
};

struct Ticket {
    int spotId; std::chrono::system_clock::time_point entryTime;
};

class IPricingStrategy {
public:
    virtual double computeFee(std::chrono::minutes duration) const = 0;
    virtual ~IPricingStrategy() = default;
};

class ParkingFloor {
public:
    std::optional<int> assignSpot(const Vehicle& v) {
        std::lock_guard<std::mutex> lock(mutex_);           // coarse-grained, per-floor (Section 4.2)
        for (auto& spot : spots_)
            if (spot.isFree() && spot.size() == v.requiredSize()) {
                spot.occupy();
                return spot.id();
            }
        return std::nullopt;                                // no fit on this floor
    }
private:
    std::vector<ParkingSpot> spots_;
    std::mutex mutex_;
};

class ParkingLot {
public:
    ParkingLot(std::shared_ptr<IPricingStrategy> pricing) : pricing_(std::move(pricing)) {} // DIP (1.6)

    std::optional<Ticket> parkVehicle(const Vehicle& v) {
        for (auto& floor : floors_)
            if (auto spotId = floor.assignSpot(v))
                return Ticket{*spotId, std::chrono::system_clock::now()};
        return std::nullopt;                                // lot full – Step 7 edge case
    }
}

```

```
double unparkVehicle(const Ticket& t) {
    auto duration = std::chrono::duration_cast<std::chrono::minutes>(
        std::chrono::system_clock::now() - t.entryTime);
    return pricing_->computeFee(duration);
}
private:
    std::vector<ParkingFloor> floors_;
    std::shared_ptr<IPricingStrategy> pricing_;
};
```

Interviewer follow-ups & deep-dive questions

- **"Now support 500 parking lots across a city, with a central availability API."** — this pushes toward sharding availability data per lot and raises whether a single in-process design still applies at all, or whether this has crossed into distributed system design territory (a good moment to explicitly name that boundary).
- **"What if two cars arrive at the exact same time for the last spot?"** — direct trigger for the Section 4 concurrency discussion.
- **"Add support for reserved/VIP spots."** — tests whether your class hierarchy can absorb a new spot subtype without modifying ParkingFloor (OCP under real pressure).
- **"The lot owner wants dynamic, demand-based pricing."** — a pure Strategy swap (2.1) if pricing was already abstracted; a real modification if it wasn't — this follow-up specifically punishes skipping Step 5.

07 Case Study: Elevator System

FREQUENCY SIGNAL: REPORTED DIRECTLY IN GLASSDOOR INTERVIEW EXPERIENCES ("DESIGN AN ELEVATOR SYSTEM USING OBJECT ORIENTED METHODOLOGIES"); INDEPENDENTLY DESCRIBED AS "ONE OF THE HARDEST COMMON OOD PROBLEMS" BECAUSE IT CENTERS ON A STATE MACHINE PLUS A SCHEDULING ALGORITHM

Problem statement, as asked

"Design an elevator system for a multi-floor building with multiple elevators — handle floor requests, assign the optimal elevator, and navigate floors, minimizing wait time."

Clarifying questions to ask: How many elevators, how many floors? Is this a passenger elevator (external up/down buttons + internal floor buttons) or freight (simpler)? Should the scheduling algorithm minimize average wait time, or guarantee fairness (no starvation)? Do elevators have a capacity limit? Out of scope: door sensors, emergency/fire-mode override, unless asked.

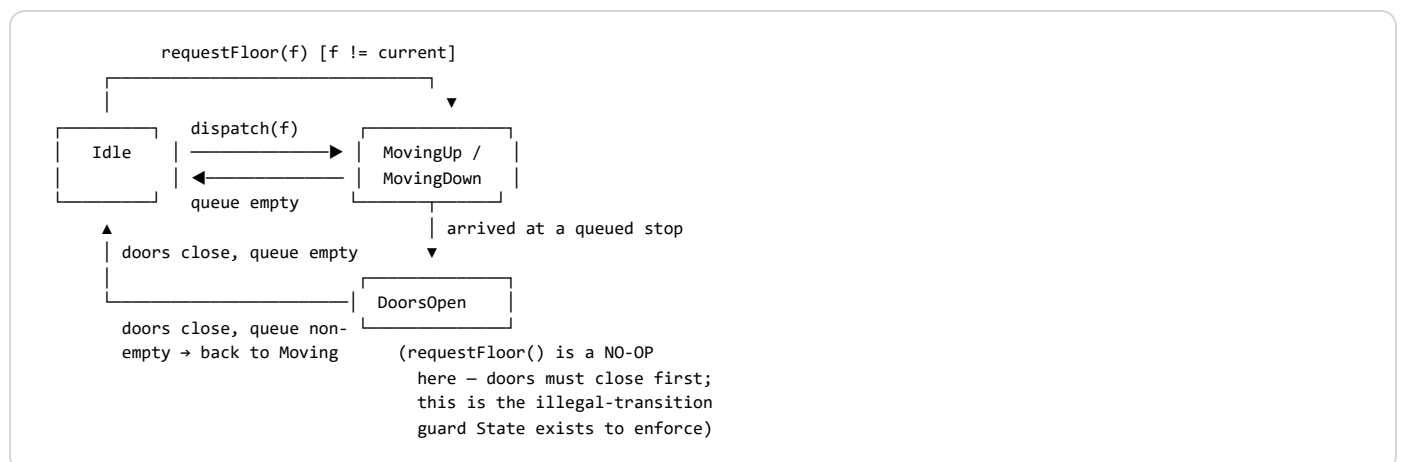
Functional & non-functional requirements

- **FR1** — A user on a floor presses up/down; the system assigns the best elevator to service that request.
- **FR2** — A user inside an elevator selects a destination floor.
- **FR3** — Each elevator tracks its own state (idle/moving-up/moving-down/doors-open) and processes its queued stops in a sensible order.
- **NFR1** — Minimize average wait time (the named non-functional requirement per search sources).
- **NFR2** — The state machine must make illegal transitions structurally impossible (can't accept a new destination while doors are open, per 2.4).

Core classes & responsibilities

CLASS	RESPONSIBILITY
ElevatorState (abstract) / Idle, MovingUp, MovingDown, DoorsOpen	State pattern (2.4) — encodes exactly which operations are legal in each state
Elevator	Owns its current state, current floor, and its own queue of destination stops; delegates requestFloor() handling to its current state object
ElevatorController	System-wide coordinator; receives external hall calls and picks which elevator services each (the scheduling algorithm lives here, not in Elevator)
Request	Value object: floor + direction (external hall call) or floor only (internal car button)
ISchedulingStrategy / NearestCarStrategy	Strategy (2.1) for elevator assignment — swappable without touching ElevatorController

State diagram (per 2.4 — "if there's a state machine, the interview is organized around it")



Key interaction: an external hall call arrives

1. User on floor 7 presses "up" →
ElevatorController::handleHallCall(Request{floor=7, dir=Up}).
2. Controller asks its ISchedulingStrategy to score every elevator (e.g. nearest elevator already moving in the same direction and about to pass floor 7 scores best; an idle elevator scores second; one moving away scores worst).
3. Controller calls Elevator::addStop(7) on the winning elevator, which delegates to its current state object — an Idle elevator transitions to MovingUp/MovingDown; a MovingUp elevator already passing floor 7 simply inserts it into its sorted stop queue.
4. Elevator "ticks" (see the callout below) toward its next queued stop; on arrival, transitions to DoorsOpen, then back to Moving* or Idle depending on whether its queue is now empty.

WHAT INTERVIEWERS LISTEN FOR

Hello Interview's own framing: whether the problem is an elevator, parking lot, vending machine, or traffic light, "you're almost always expected to build a simulation... abstracting away the hardware and controlling time yourself with a step() or tick() method" rather than real timers — naming this pattern explicitly is a strong, cheap signal.

Public API (key signatures)

```
class ElevatorController {
public:
    void handleHallCall(int floor, Direction dir);
    void handleCarButton(int elevatorId, int destinationFloor);
    void step();    // advances simulated time by one unit
};
```

Design choices, trade-offs & extension scenarios

- **Where does the stop queue live?** On the Elevator itself (each elevator manages its own sorted set of pending stops), not the controller — this keeps the controller's job purely "which elevator," not "how does this elevator sequence its stops," a direct SRP application (1.4).
- **Extension: scale to 50 elevators, 100 floors.** The scheduling strategy's cost per hall call matters now — a naive $O(\text{elevators})$ linear scan is fine at small scale but should be named as a bottleneck at scale, opening a discussion of maintaining per-elevator sorted stop sets for $O(\log n)$ insertion.
- **Extension: capacity limits.** An elevator at max capacity must reject new hall-call assignments from the scheduler even if it's otherwise the "best" candidate — this is a clean addition to the ISchedulingStrategy's scoring function, not a change to the state machine.

Concurrency considerations

Multiple hall-call button presses can arrive concurrently from different floors while an elevator's state is being read by the scheduler. The default: each Elevator owns its own mutex guarding its state + stop queue; the controller takes a consistent snapshot of all elevator states before scoring, to avoid scoring against a state that changes mid-computation.

Test cases & what to mock

- Idle elevator receives a hall call → transitions to Moving* toward the correct floor.
 - Elevator in DoorsOpen state receives requestFloor() → no-op / queued, not immediately actioned (the illegal-transition guard).
 - Two hall calls in opposite directions from the same floor → both eventually serviced, neither dropped.
 - Scheduling strategy is **mocked/faked** in Elevator-level unit tests, so state-machine correctness is tested independently of assignment-quality; a separate test suite validates the strategy's scoring logic in isolation.
-

Minimal C++ code skeleton

```

enum class Direction { Up, Down, None };

class Elevator; // fwd decl

class ElevatorState {
public:
    virtual void requestFloor(Elevator& ctx, int floor) = 0;
    virtual void step(Elevator& ctx) = 0;
    virtual ~ElevatorState() = default;
};

class Elevator {
public:
    void setState(std::unique_ptr<ElevatorState> s) { state_ = std::move(s); }
    void addStop(int floor) { std::lock_guard<std::mutex> l(mutex_); stops_.insert(floor); }
    void requestFloor(int floor) { state_->requestFloor(*this, floor); }
    void step() { state_->step(*this); }
    int currentFloor() const { return currentFloor_; }
private:
    int currentFloor_ = 0;
    std::set<int> stops_;
    std::unique_ptr<ElevatorState> state_;
    std::mutex mutex_;
};

class DoorsOpenState : public ElevatorState {
public:
    void requestFloor(Elevator&, int) override { /* no-op - the illegal-transition guard */ }
    void step(Elevator& ctx) override { /* close doors, transition to Idle or Moving* */ }
};

class ISchedulingStrategy {
public:
    virtual Elevator* selectElevator(
        std::vector<std::unique_ptr<Elevator>>& elevators, int floor, Direction dir) = 0;
    virtual ~ISchedulingStrategy() = default;
};

class ElevatorController {
public:
    ElevatorController(std::shared_ptr<ISchedulingStrategy> strategy)
        : strategy_(std::move(strategy)) {} // DIP (1.6)
    void handleHallCall(int floor, Direction dir) {
        if (auto* e = strategy_->selectElevator(elevators_, floor, dir))
            e->addStop(floor);
    }
private:
    std::vector<std::unique_ptr<Elevator>> elevators_;
    std::shared_ptr<ISchedulingStrategy> strategy_;
};

```

Interviewer follow-ups & deep-dive questions

- **"Change the scheduling to guarantee no request waits more than N seconds."** — pushes from pure nearest-car optimization toward an aging/fairness-aware strategy, testing

whether Strategy (2.1) was actually decoupled from Controller.

- **"Add a VIP/executive floor that gets priority."** — tests whether priority can be added to the scheduling strategy's scoring function alone, without touching the state machine.
- **"What happens if an elevator breaks down mid-transit?"** — a new state (OutOfService) plus a re-dispatch of its queued stops to other elevators; tests state-machine extensibility directly.
- **"How would you test this without waiting for real elevator movement?"** — the expected answer is exactly the `step()`/simulated-time pattern named above.

08 Case Study: LRU Cache

FREQUENCY SIGNAL: NAMED DIRECTLY AMONG "THE MOST FREQUENTLY ASKED PROBLEMS" ALONGSIDE PARKING LOT AND RATE LIMITER; SITS IN THE "TECHNICAL SYSTEMS" CATEGORY SEARCH SOURCES DISTINGUISH FROM GAMES AND PRODUCT SIMULATIONS; THE THREAD-SAFETY FOLLOW-UP IS DOCUMENTED AS A NEAR-DEFAULT DEEP DIVE, NOT AN EDGE CASE

Problem statement, as asked

"Design a cache with a fixed capacity that supports $O(1)$ get and put, evicting the least-recently-used entry when full."

Clarifying questions to ask: Does get count as a "use" that refreshes recency (yes, by definition of LRU)? Fixed capacity set at construction, or resizable? Single-threaded to start, with thread safety as a possible follow-up? What should put do on an existing key — update value and refresh recency?

Functional & non-functional requirements

- **FR1** — `put(key, value)`: insert or update; if at capacity and inserting a new key, evict the least-recently-used entry first.
- **FR2** — `get(key)`: return the value if present (and mark it most-recently-used), or a "not found" signal.
- **NFR1** — Both operations must be $O(1)$ average time — this single constraint drives the entire data-structure choice (Step 6 of the framework).
- **NFR2** (common follow-up) — thread-safe under concurrent get/put from multiple threads.

Core classes & responsibilities

CLASS	RESPONSIBILITY
Node	Intrusive doubly-linked-list node holding key, value, prev/next pointers
LRUCache	Owns a hash map (key → Node*) for O(1) lookup, plus a doubly-linked list for O(1) recency reordering — the two structures together are the entire mechanism

This is deliberately the smallest class list in this guide — LRU Cache is a data-structure problem wearing LLD clothing; the "design" content is almost entirely in *why* a hash map alone (no recency order) or a linked list alone (no O(1) lookup) each fail NFR1 individually, and only the combination satisfies it.

Structure diagram

```

hashMap: { key → Node* }

Doubly-linked list, ordered by recency (head = most recent, tail = least recent):

    head ⇄ [k3|v3] ⇄ [k1|v1] ⇄ [k2|v2] ⇄ tail
           (MRU)                (LRU – evicted first)

get(k1):  1) hashMap[k1] → O(1) node lookup
          2) unlink node, re-insert right after head → O(1)

put(new key) when at capacity:
    1) evict tail.prev (the LRU node) – unlink it, erase from hashMap
    2) insert new node right after head

```

Public API (key signatures)

```

template<typename K, typename V>
class LRUCache {
public:
    explicit LRUCache(size_t capacity);
    std::optional<V> get(const K& key);
    void put(const K& key, V value);
};

```

Design choices, trade-offs & extension scenarios

- **Why a doubly-linked list, not a singly-linked one?** Removing a node from the middle (on a get that promotes it) requires O(1) access to *both* neighbors to patch the links — a singly-linked list only gives you the next pointer, forcing an O(n) scan to find the previous node.
- **Extension: LFU (Least Frequently Used) instead of LRU.** This is a genuinely different eviction policy, and the clean answer is Strategy (2.1): extract an IEvictionPolicy interface the cache delegates to, so swapping LRU for LFU doesn't touch the cache's core get/put logic — a strong, expected follow-up answer.

- **Extension: TTL-based expiry alongside LRU.** Requires storing an expiry timestamp per node and checking it on get (lazy expiry) — a good moment to discuss lazy vs. active (background sweep) expiration trade-offs.
-

Concurrency considerations

Per Section 4.2/4.3: the default, safe answer is one coarse-grained `std::mutex` guarding both the map and the list together, since a get mutates the list (promoting a node) even though it "sounds" read-only — this is the specific trap worth naming unprompted. The escalation, only if pushed for high-read-throughput: a `ConcurrentHashMap`-equivalent for the key lookup paired with a concurrent-safe list structure, accepting the real complexity cost named in 4.3.

Test cases & what to mock

- put beyond capacity evicts the correct (least-recently-used, not least-recently-inserted) entry — this specific test catches the most common LRU implementation bug: forgetting that get also refreshes recency.
 - get on a missing key returns "not found" without mutating any state.
 - put on an existing key updates the value and moves it to most-recently-used, without changing the cache's size.
 - Nothing external to mock here — this is a self-contained data structure; the only "test double" concern is a fake/deterministic clock if TTL expiry is added.
-

Minimal C++ code skeleton

```

template<typename K, typename V>
class LRUCache {
public:
    explicit LRUCache(size_t cap) : capacity_(cap) {}

    std::optional<V> get(const K& key) {
        std::lock_guard<std::mutex> lock(mutex_);           // get() mutates order – needs the lock too
        auto it = map_.find(key);
        if (it == map_.end()) return std::nullopt;
        list_.splice(list_.begin(), list_, it->second);      // O(1) move-to-front
        return it->second->value;
    }

    void put(const K& key, V value) {
        std::lock_guard<std::mutex> lock(mutex_);
        auto it = map_.find(key);
        if (it != map_.end()) {
            it->second->value = value;
            list_.splice(list_.begin(), list_, it->second);
            return;
        }
        if (map_.size() >= capacity_) {
            auto& lru = list_.back();                         // tail = least-recently-used
            map_.erase(lru.key);
            list_.pop_back();
        }
        list_.push_front({key, value});
        map_[key] = list_.begin();
    }

private:
    struct Entry { K key; V value; };
    size_t capacity_;
    std::list<Entry> list_;                                  // front = MRU, back = LRU
    std::unordered_map<K, typename std::list<Entry>::iterator> map_;
    std::mutex mutex_;
};

```

Interviewer follow-ups & deep-dive questions

- **"Make it thread-safe."** — the single most commonly reported LRU Cache follow-up; expect to discuss coarse vs. fine-grained locking explicitly (Section 4.2).
 - **"Now make gets lock-free."** — the senior-track escalation; the honest answer names the real complexity trade-off (4.3) rather than reflexively reaching for lock-free structures.
 - **"Swap the eviction policy to LFU."** — tests the Strategy extraction named above.
 - **"What if the cache needs to be shared across multiple processes/machines?"** — a deliberate scope-check: this crosses from LLD into distributed-caching system design (Redis-style), and naming that boundary explicitly is itself a good answer.
-

09 Case Study: Rate Limiter

FREQUENCY SIGNAL: GROUPED WITH LRU CACHE AND PARKING LOT AS ONE OF "THE MOST FREQUENTLY ASKED PROBLEMS"; EXPLICITLY NAMED AS ONE OF "TWO SMALL, BATTLE-TESTED PRIMITIVES THAT EVERY BACKEND SERVICE NEEDS," ALONGSIDE LRU CACHE — BOTH INDEPENDENTLY CONFIRMED AS A THREAD-SAFETY-HEAVY FOLLOW-UP TRACK

Problem statement, as asked

"Design a rate limiter that allows at most N requests per client per time window, rejecting requests beyond that limit."

Clarifying questions to ask: Per-client (per API key/user ID) or global? Fixed window, sliding window, or token-bucket semantics — does bursting matter, or is smooth, steady-rate throttling required? In-memory single-instance, or does this need to work across multiple servers sharing state (out of scope unless asked — that's a distributed-systems escalation, same boundary as LRU Cache's follow-up)?

Functional & non-functional requirements

- **FR1** — `allowRequest(clientId)`: return true/false for whether this request is within the client's current limit.
- **FR2** — Support configuring different limits per client tier (free vs. paid).
- **NFR1** — $O(1)$ or $O(\log n)$ per check — this is called on the hot path of every incoming request.
- **NFR2** — Thread-safe under high concurrent request volume from many clients simultaneously.

Core classes & responsibilities

CLASS	RESPONSIBILITY
IRateLimitAlgorithm / TokenBucket , SlidingWindowLog , FixedWindowCounter	Strategy (2.1) — the actual throttling algorithm, swappable per client tier
RateLimiter	Top-level coordinator; maps <code>clientId</code> → that client's algorithm instance/state, delegates <code>allowRequest</code>
RateLimitConfig	Value object: limit, window size, algorithm choice — per tier

Algorithm comparison (the core design decision for this problem)

ALGORITHM	HOW IT WORKS	TRADE-OFF
Fixed Window Counter	Count requests in the current fixed time slice (e.g. this minute); reset each slice	Simplest, $O(1)$ memory per client, but allows up to $2\times$ the limit right across a window boundary (burst at 23:59:59 + burst at 00:00:00)
Sliding Window Log	Store a timestamp per request; count how many fall within the trailing window	Precise, no boundary burst, but $O(\text{limit})$ memory per client and $O(\text{limit})$ work per check unless pruned carefully
Token Bucket	Tokens refill at a fixed rate up to a max; each request consumes one token	Naturally allows controlled bursting (spend saved-up tokens) while enforcing a steady long-run rate — the most commonly recommended default

Key interaction: `allowRequest(clientId)`

1. RateLimiter looks up (or lazily creates) the `IRateLimitAlgorithm` instance for this client.
2. Delegates to that algorithm's `tryConsume()` — for Token Bucket: compute tokens accrued since last check (elapsed time $\times$ refill rate), cap at bucket size, then attempt to deduct one.
3. Returns true/false; a false result is what the caller (an API gateway, typically) turns into an HTTP 429.

Public API (key signatures)

```
class RateLimiter {
public:
    RateLimiter(std::shared_ptr<IRateLimitAlgorithmFactory> factory);
    bool allowRequest(const std::string& clientId);
};
```

Design choices, trade-offs & extension scenarios

- **Why Token Bucket as the default recommendation?** It's the only one of the three that cleanly separates "average sustained rate" from "permitted burst size" as two independent, tunable parameters — matching how real APIs are actually throttled (occasional bursts tolerated, sustained abuse blocked).
- **Extension: different limits per endpoint, not just per client.** Key the algorithm-instance map by `(clientId, endpoint)` composite instead of `clientId` alone — a small change if the map key was already abstracted, a real one if it wasn't (tests whether Step 4/5 were done generically).
- **Extension: distributed rate limiting across multiple server instances.** The in-memory per-process map no longer works — state must move to a shared store (Redis, typically, using its atomic INCR+TTL or a Lua script for token-bucket logic). This is the standard,

expected escalation into system-design territory, and naming it explicitly (rather than trying to hand-wave a multi-server in-memory solution) is the strong answer.

Concurrency considerations

Per Section 4: many threads check the *same* client's bucket concurrently (a hot client hammering the API) — this needs an atomic check-and-consume, not just a lock around the read. A per-client mutex (fine-grained, since different clients never contend with each other) guarding that client's token count and last-refill timestamp is the natural design — a single global lock across all clients would serialize unrelated clients' requests unnecessarily, a real and easy-to-articulate case *for* fine-grained locking rather than the usual coarse-grained default.

Test cases & what to mock

- Exactly N requests allowed within a window; the (N+1)th rejected.
 - After the window/refill period elapses, capacity is available again — this requires a **fake/injectable clock** (real time in a unit test is both slow and flaky) so the test can advance time deterministically rather than sleeping.
 - Token Bucket specifically: verify a burst up to bucket size succeeds even if requests arrive faster than the refill rate, and that sustained requests beyond the refill rate are throttled.
 - Concurrent requests from the same client right at the limit boundary → exactly N succeed, not N+1 (the test that exercises the atomic-consume requirement above).
-

Minimal C++ code skeleton

```

class TokenBucket {
public:
    TokenBucket(double capacity, double refillPerSecond)
        : capacity_(capacity), tokens_(capacity), refillRate_(refillPerSecond),
          lastRefill_(std::chrono::steady_clock::now()) {}

    bool tryConsume() {
        std::lock_guard<std::mutex> lock(mutex_);           // per-client lock (fine-grained by design)
        refill();
        if (tokens_ < 1.0) return false;
        tokens_ -= 1.0;
        return true;
    }
private:
    void refill() {
        auto now = std::chrono::steady_clock::now();
        double elapsedSec = std::chrono::duration<double>(now - lastRefill_).count();
        tokens_ = std::min(capacity_, tokens_ + elapsedSec * refillRate_);
        lastRefill_ = now;
    }
    double capacity_, tokens_, refillRate_;
    std::chrono::steady_clock::time_point lastRefill_;
    std::mutex mutex_;
};

class RateLimiter {
public:
    explicit RateLimiter(double capacity, double refillPerSecond)
        : capacity_(capacity), refillRate_(refillPerSecond) {}

    bool allowRequest(const std::string& clientId) {
        std::shared_ptr<TokenBucket> bucket;
        {
            std::lock_guard<std::mutex> lock(mapMutex_);     // guards map structure only, briefly
            auto it = buckets_.find(clientId);
            if (it == buckets_.end())
                it = buckets_.emplace(clientId,
                                       std::make_shared<TokenBucket>(capacity_, refillRate_)).first;
            bucket = it->second;
        }
        return bucket->tryConsume();                         // per-client lock, not the map lock
    }
private:
    double capacity_, refillRate_;
    std::unordered_map<std::string, std::shared_ptr<TokenBucket>> buckets_;
    std::mutex mapMutex_;
};

```

Interviewer follow-ups & deep-dive questions

- **"Make this work across a fleet of servers behind a load balancer."** — the expected pivot to a shared store (Redis) named above; a strong candidate raises this unprompted before being asked.
- **"Different clients need different algorithms — some need strict fixed windows, others need bursting."** — directly validates whether `IRateLimitAlgorithm` was actually

abstracted as Strategy, or hardcoded.

- **"How do you avoid a memory leak from tracking millions of clients that only ever send one request?"** — a real production concern: an LRU-style eviction (Case Study 8) of stale per-client bucket entries, tying two case studies together.
- **"What HTTP status/headers would you return on rejection?"** — tests whether the candidate connects the LLD component to its actual place in a real API (429, Retry-After header).

10 Case Study: Splitwise (Expense Sharing)

FREQUENCY SIGNAL: NAMED DIRECTLY ALONGSIDE PARKING LOT, ELEVATOR, AND CAR RENTAL AS ONE OF THE "POPULAR PRODUCTS" CATEGORY OF LLD PROBLEMS THAT RECURS ACROSS AGGREGATED QUESTION LISTS

Problem statement, as asked

"Design a system where a group of users can log shared expenses, split them (equally, by exact amount, or by percentage), and compute who owes whom how much, minimizing the number of settlement transactions."

Clarifying questions to ask: Which split types are in scope — equal, exact amount, percentage, or all three? Is "simplify debts" (minimizing total transactions) required, or is showing raw pairwise balances acceptable? Single currency? Are groups persistent entities, or is every expense just tagged with a set of participants?

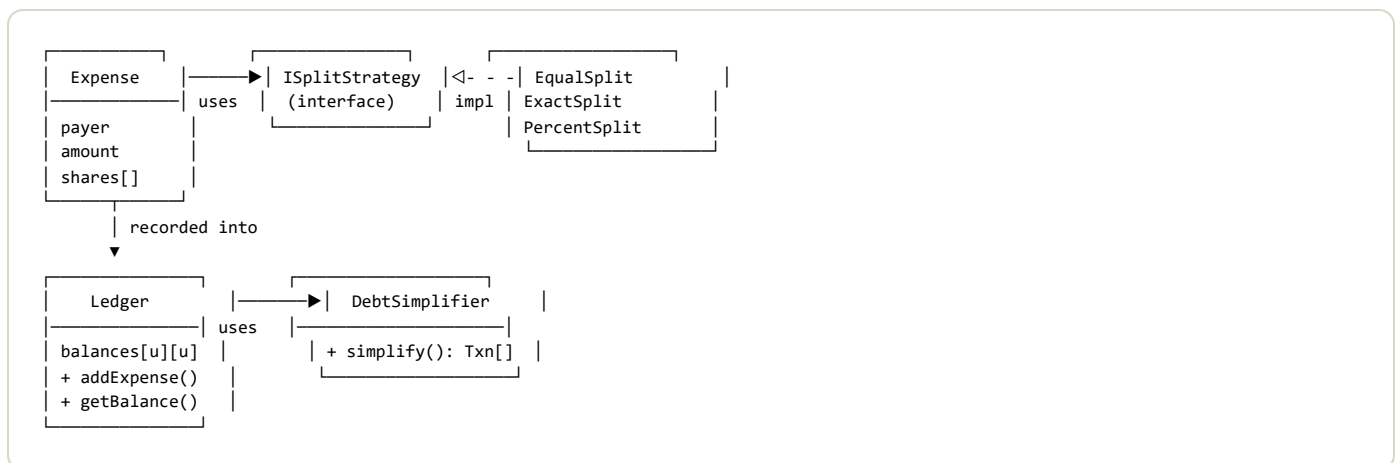
Functional & non-functional requirements

- **FR1** — Add an expense: one payer, a list of participants, a split strategy (equal/exact/percentage), validate the split sums correctly to the total.
- **FR2** — Query balances: how much does user A owe user B, across all their shared expenses.
- **FR3** — Simplify/settle: compute a minimal set of transactions that zeroes out all balances within a group.
- **NFR1** — Adding a new split type must not require modifying the expense-recording logic (OCP, direct Strategy application).
- **NFR2** — Balance computation must be correct to the cent — floating-point rounding errors on percentage splits are a real, citable edge case.

Core classes & responsibilities

CLASS	RESPONSIBILITY
User	Identity only — id, name
Expense	Immutable record: payer, total amount, list of (user, owedShare) pairs, timestamp
ISplitStrategy / EqualSplit, ExactSplit, PercentSplit	Strategy (2.1) — computes each participant's owed share from the raw input; the only place split-type logic lives
Ledger / BalanceSheet	Maintains the running pairwise net-balance graph (who owes whom) as expenses are added
DebtSimplifier	Given the balance graph, computes a minimal transaction set to settle it (a distinct algorithmic concern, kept separate from Ledger per SRP)

Class diagram (simplified notation)



Key interaction: addExpense with a percentage split

1. Caller supplies payer, total amount, participants, and per-participant percentages.
2. `PercentSplit::computeShares()` validates percentages sum to 100 (Step 7 edge case), then computes each share as $\text{amount} * \text{pct} / 100$, rounding the *last* participant's share to absorb any floating-point remainder so the sum exactly equals the total (a specific, citable correctness technique for NFR2).
3. `Ledger::addExpense()` updates the pairwise balance graph: for each non-payer participant, increase "participant owes payer" by that participant's share.

Public API (key signatures)

```

class ISplitStrategy {
public:
    virtual std::unordered_map<UserId, double> computeShares(
        double totalAmount, const std::vector<UserId>& participants,
        const std::unordered_map<UserId, double>& inputs) const = 0;
};

class Ledger {
public:
    void addExpense(const Expense& e);
    double getBalance(UserId a, UserId b) const; // positive: b owes a
    std::vector<Transaction> simplifyDebts() const;
};

```

Design choices, trade-offs & extension scenarios

- **Why a pairwise balance map instead of replaying every Expense on every query?** $O(1)$ balance lookup vs. $O(\text{expenses})$ — a direct NFR-driven data-structure choice (Step 6); the trade-off is that Ledger must be updated incrementally and correctly on every addExpense, rather than being a pure derived view.
- **Debt simplification algorithm.** The standard greedy approach: repeatedly match the participant with the largest net-owed amount against the participant with the largest net-owes amount, settling the smaller of the two, until all balances are zero — this minimizes transaction count but is worth naming as a greedy heuristic, not a proven-optimal minimum (the true minimum-transaction problem is more complex; greedy is the accepted, expected interview answer).
- **Extension: multi-currency support.** Requires an explicit exchange-rate boundary — an ICurrencyConverter injected (DIP) wherever amounts are compared, since balances can no longer be summed as raw doubles across currencies. A good test of whether the candidate over-simplified by assuming single-currency without stating it as an assumption in Step 2.

Concurrency considerations

Two group members adding expenses involving the same pair of users concurrently must not lose an update to the shared balance entry — a per-pair (or per-group) lock around `Ledger::addExpense`'s read-modify-write of the balance map is the coarse-grained default; this is a lower-priority concurrency discussion than Elevator/LRU Cache/Rate Limiter per search sources, since Splitwise's core interview value is in the split-strategy modeling and the simplification algorithm, not throughput.

Test cases & what to mock

- Equal split among 3 users on an amount not evenly divisible by 3 (e.g. \$10.00) → shares sum exactly to \$10.00, with the rounding remainder assigned deterministically (e.g. to the first

participant), not silently lost.

- Percentage split with percentages summing to 100 succeeds; summing to 99 or 101 is rejected (Step 7).
- `getBalance(A, B)` after several expenses in both directions nets correctly (A paid for B once, B paid for A twice → correct signed net).
- `simplifyDebts()` on a 3-person cycle (A owes B, B owes C, C owes A, all equal amounts) → correctly resolves to zero transactions, not three — the single best test case for validating the simplification logic actually works rather than just running.
- No external dependencies to mock in the core ledger; a fake `ICurrencyConverter` if that extension is added.

Minimal C++ code skeleton

```
class EqualSplit : public ISplitStrategy {
public:
    std::unordered_map<UserId, double> computeShares(
        double total, const std::vector<UserId>& participants,
        const std::unordered_map<UserId, double>&) const override {
        std::unordered_map<UserId, double> shares;
        double base = std::floor((total / participants.size()) * 100) / 100;
        double assigned = 0;
        for (size_t i = 0; i < participants.size(); ++i) {
            bool isLast = (i == participants.size() - 1);
            shares[participants[i]] = isLast ? (total - assigned) : base; // last absorbs rounding
            assigned += base;
        }
        return shares;
    }
};

class Ledger {
public:
    void addExpense(const Expense& e) {
        std::lock_guard<std::mutex> lock(mutex_);
        for (const auto& [user, share] : e.shares) {
            if (user == e.payer) continue;
            balances_[user][e.payer] += share; // user owes payer `share` more
        }
    }

    double getBalance(UserId a, UserId b) const {
        double aOwesB = balances_.count(a) && balances_.at(a).count(b) ? balances_.at(a).at(b) : 0;
        double bOwesA = balances_.count(b) && balances_.at(b).count(a) ? balances_.at(b).at(a) : 0;
        return bOwesA - aOwesB; // positive: b owes a, net
    }

private:
    std::unordered_map<UserId, std::unordered_map<UserId, double>> balances_;
    std::mutex mutex_;
};
```

Interviewer follow-ups & deep-dive questions

- **"A user disputes an expense and it needs to be edited after the fact."** — tests whether Expense being modeled as immutable was a real constraint or an oversight; the clean answer is reversing the old expense's ledger effect before applying the edited one, not mutating balances in place.
 - **"Support recurring expenses (e.g. monthly rent)."** — a scheduling concern layered on top of the existing addExpense flow; tests whether the candidate keeps expense-creation and expense-scheduling as separate concerns (SRP again).
 - **"Add multi-currency support."** — directly the extension scenario above; a strong candidate flags the single-currency assumption during Step 2 rather than being surprised by this follow-up.
 - **"Prove your debt-simplification algorithm is correct / minimal."** — the expected honest answer: greedy largest-creditor-vs-largest-debtor matching is a well-known good heuristic, not a proven minimum-transaction-count solution — naming that distinction is itself the strong answer, not a weakness to hide.
-

11 Interviewer Follow-ups, Evaluation Rubric & Common Mistakes

Each case study in Sections 6–10 already lists its own specific follow-ups and a "strong candidate says / red flag" pair. This section consolidates the pattern that repeats across all five, into one general rubric, before the detailed mistakes list below.

Consolidated evaluation rubric

A strong candidate...

- Asks clarifying questions before drawing anything (Section 0)
- Names *why* a class/pattern was chosen, unprompted
- Extends the existing design when the interviewer changes a requirement, rather than restarting
- Mentions edge cases and tests before being asked
- Says "I'm not sure, let me think out loud" instead of going silent when stuck

A red flag looks like...

- Starting to code/draw immediately, no questions asked
- A single class doing several unrelated jobs (SRP violation, 1.4)
- Adding a pattern or abstraction with no concrete need for it yet (YAGNI, 1.7)
- The whole design falling apart when one requirement changes
- Long silence with no explanation of what's being considered

Then, the specific mistakes list

MISTAKE	WHY IT READS AS A WEAK SIGNAL	HOW TO RECOVER IF YOU CATCH YOURSELF DOING IT
Over-engineering — adding databases, caching layers, or threading nobody asked for	"You're not building production software, you're demonstrating design thinking" — unnecessary complexity obscures whether the core design is even correct	Explicitly say "I'll keep this in-memory/single-threaded for now and layer in X if we have time" — name the simpler choice as deliberate, not lazy
Skipping requirements clarification	Reads as not knowing the framework at all; the rest of the design is built on assumptions the interviewer never confirmed	Even mid-design, it's recoverable to say "actually, let me confirm an assumption I made earlier" — better late than never
Poor time management — diving into code before the structure is agreed, or over-polishing early details	Both directions burn the clock before a complete design is shown at all	State your plan out loud upfront: "I'll spend a few minutes on requirements, then the class diagram, then code the trickiest method" — this itself is a signal of structure
Silent design choices — arriving at a diagram without narrating why	"Interviewers can't read your mind" — an unexplained choice can't be evaluated, and looks like it wasn't a choice at all	Narrate every non-obvious decision the moment you make it, not retroactively
SRP violations via "god classes" — one class doing allocation, payment, and printing	The single most-cited concrete anti-pattern; signals the candidate hasn't internalized SRP beyond the definition	Ask yourself "what's the one reason this class would change?" for every class before finalizing
Overusing inheritance where composition fits better	Produces rigid hierarchies that break under the very first follow-up (Section 1.2's Bird/Penguin problem)	Default to composition; justify inheritance explicitly when you do reach for it
Ignoring edge cases and error handling	"Great candidates differentiate themselves by thinking of edge cases before concluding" — its absence is conspicuous by comparison	Before declaring the design done, explicitly ask aloud: "what happens on invalid input, on capacity exceeded, on a not-found lookup?"
Consumed material but can't apply it live	Passive familiarity with patterns/principles doesn't transfer under interview pressure without rehearsal	Not recoverable mid-interview — the fix is practicing full case studies beforehand, which is exactly what Sections 6–10 are for

IF YOU GET GENUINELY STUCK

Narrate the stuck point itself rather than going silent — "I'm not sure whether to model this as inheritance or composition; let me think through the trade-off out loud" is a recoverable, even positive moment. Silence is the actual failure mode, not uncertainty.

12 Rapid Revision Sheet & Glossary

A last-minute-before-the-interview pass — everything above, compressed.

- 1. Clarify requirements (5–8 min, always)
- 2. State assumptions out loud
- 3. Use cases + NFRs
- 4. Entities + SRP-driven responsibilities
- 5. Interfaces + pattern selection
- 6. Data structures + Big-O
- 7. Error handling + edge cases
- 8. Concurrency (only if relevant)
- 9. Tests + extensibility notes

SOLID, ONE LINE EACH

- **Single Responsibility** — one reason to change
- **Open/Closed** — extend, don't modify
- **Liskov Substitution** — subtypes must honor the base's contract
- **Interface Segregation** — no fat interfaces
- **Dependency Inversion** — depend on abstractions, inject concretes

PATTERN → SIGNAL THAT YOU NEED IT

- **Strategy** — "this behavior/algorithm varies" (pricing, eviction, throttling)
- **State** — "this entity has named states with different legal ops per state"
- **Observer** — "multiple independent things must react to one change"
- **Factory** — "object creation logic is non-trivial or type-dependent"
- **Singleton** — "exactly one coordinating instance must exist, for real reasons"

CASE STUDY → CORE DATA STRUCTURE

- **Parking Lot** — spots grouped by size (hash map of vectors/heaps)
- **Elevator** — per-elevator sorted stop set + State machine
- **LRU Cache** — hash map + doubly-linked list
- **Rate Limiter** — per-client Token Bucket (timestamp + counter)
- **Splitwise** — pairwise balance map + greedy debt-simplification

CONCURRENCY DEFAULT

- Start coarse-grained (one lock), always
- Name the specific race condition that would occur without it
- Escalate to fine-grained only if asked for throughput under contention
- Fine-grained raises deadlock risk — say so if you escalate

Glossary

Low-Level Design (LLD)

Object-oriented design carried far enough to produce compilable classes and method signatures.

SOLID

Five principles (SRP, OCP, LSP, ISP, DIP) for maintainable, extensible class design.

Strategy pattern

Encapsulates an interchangeable algorithm/behavior behind a common interface.

State pattern

Encodes an entity's states as separate classes, making illegal transitions structurally hard to express.

Observer pattern

Lets multiple subscribers react to a state change without the subject knowing who they are.

Factory pattern

Centralizes object creation behind one method/class.

Composition over inheritance

Default preference for "has-a" delegation over "is-a" hierarchies, for flexibility.

Coarse-grained lock

One lock guarding an entire structure — simple, safe, less concurrent.

Fine-grained lock

Multiple, narrower locks — more concurrency, more deadlock risk.

Race condition

A bug where the outcome depends on the timing/interleaving of concurrent operations.

Dependency injection

Supplying a class's collaborators (often via constructor) rather than having it construct them internally — enables testing with fakes.

Token Bucket

A rate-limiting algorithm where tokens refill at a fixed rate and each request consumes one, allowing controlled bursts.

Sources consulted

Compiled from current interview-report and practitioner sources as of September 2026. Where this guide says a question or problem is "commonly asked" or "frequently reported," that reflects language or aggregated data found directly in the cited source, not an invented popularity claim.

GLASSDOOR & INTERVIEW-EXPERIENCE FORUMS

[Glassdoor — Wayfair SDE-II: automated parking lot system](#)

[Glassdoor — "Design an elevator system using object oriented methodologies"](#)

[Blind — "Were you asked OOD in your FAANG interview?"](#)

[Blind — Low level design mock](#)

[Blind — "Any good material on LLD"](#)

GITHUB HANDBOOKS & REPOSITORIES

[ashishps1/awesome-low-level-design](#)

[kumaransg/LLD — curated LLD questions asked at major tech companies](#)

[hardihood/LLD](#)

GEEKSFORGEEKS & STRUCTURED PREP GUIDES

[GeeksforGeeks — How to Prepare for Low-Level Design Interviews](#)

[GeeksforGeeks — Top Low-Level Design Interview Questions](#)

[GeeksforGeeks — Design Patterns Interview Questions](#)

[GeeksforGeeks — Elevator System Low-Level Design](#)

[InterviewBit — Low-Level Design Interview Questions](#)

PRACTITIONER & ENGINEERING BLOGS

[Hello Interview — Low Level Design in a Hurry \(framework + delivery\)](#)

[Hello Interview — Parking Lot Low Level Design](#)

[Hello Interview — Elevator Low Level Design](#)

[Grind Engineer — "LLD Was HARD Until I Learned These 12 Design Patterns"](#)

[Medium — Cracking the Low Level Design \(LLD\) Interview](#)

[Medium — Thread-Safe LRU Cache: Design and Implementation](#)

[Adaface — SOLID Interview Questions](#)

LLD Foundations Handbook · Beginner-to-Advanced LLD/OOD Edition · Code skeletons in C++. Use the "Save as PDF" button (top of page) to export via your browser's print dialog for offline study. A companion single-file LaTeX edition of this same content is provided as a separate .tex file.